

Informe de ingeniería de software

Sistema de gestión para gimnasios de boxeo · Piloto: Diamante Boxing Club
Fundamentos teóricos, arquitectura, datos, seguridad, calidad y operación

Contenido

1. Resumen ejecutivo
2. Fundamentos teóricos considerados
3. Requisitos y atributos de calidad
4. Arquitectura
5. Módulos y modelo del dominio
6. Modelo de datos y persistencia
7. Seguridad y privacidad
8. Reglas del negocio y fundamentos del dominio deportivo
9. Interfaz y experiencia de uso
10. Calidad y pruebas
11. Infraestructura, despliegue y operación
12. Gestión de la configuración y versiones
13. Decisiones, riesgos y deuda técnica
14. Glosario

Estado: versión v0.11 en producción · 4 de noviembre de 2026. Complementa al Plan del sistema (docs/Plan_Sistema_Gimnasio.pdf).

1. Resumen ejecutivo

El sistema es una aplicación web, pensada primero para celular, que administra un gimnasio de boxeo de punta a punta: accesos y roles, horario de coaches, registro y membresías de clientes, pagos y comisiones, cuadre mensual, evaluación física por pilares, biblioteca de ejercicios y planes de trabajo, novedades y captación (clase de prueba y solicitudes de coach). Nació para un gimnasio piloto, pero se diseñó **multi-gimnasio** desde el inicio para ofrecerse como servicio, con los datos de cada gimnasio aislados en la base de datos.

Aspecto	Decisión
Estilo arquitectónico	Arquitectura limpia por módulos (contextos acotados), puertos y adaptadores
Lenguaje y marco	TypeScript estricto · Next.js 15 (App Router, React 19, acciones de servidor)
Base de datos	PostgreSQL administrado (Neon) con seguridad a nivel de fila (RLS)
Ejecución	Contenedor en Google Cloud Run (sin servidores que mantener), región us-central1
Calidad	146 pruebas en 23 archivos (dominio, casos de uso e integración con Postgres real embebido), ESLint con regla de dependencia, tipos estrictos
Operación	Publicación con un solo comando, versión semántica, reversión inmediata por revisión y respaldo nocturno
Tamaño	≈12,700 líneas de código fuente, 7 módulos de negocio, 13 migraciones, 55 tablas

2. Fundamentos teóricos considerados

Esta sección reúne los marcos teóricos que guiaron las decisiones, y dónde se aplican en el sistema.

2.1 Arquitectura limpia (Robert C. Martin)

El software se organiza en capas concéntricas y la **regla de dependencia** dice que el código sólo puede depender hacia adentro: las reglas del negocio no conocen la base de datos, el marco web ni la interfaz. Así las reglas se prueban sin infraestructura y la tecnología se puede cambiar sin reescribirlas.

- **Dominio** (entidades y reglas puras) → **Aplicación** (casos de uso y puertos) → **Infraestructura** (Postgres, cifrado) → **Marco** (Next.js).
- La regla se verifica automáticamente: ESLint prohíbe que el dominio importe casos de uso, infraestructura, React, Next o el cliente de Postgres, y que los casos de uso importen infraestructura.

2.2 Arquitectura hexagonal: puertos y adaptadores (Alistair Cockburn)

Cada caso de uso declara lo que necesita del exterior como una interfaz (*puerto*): repositorios, reloj, generador de identificadores, autorización, auditoría, cifrador. La infraestructura aporta los *adaptadores* (por ejemplo *SolicitudesPostgres*). En pruebas se sustituyen por implementaciones en memoria o por un reloj fijo, lo que vuelve deterministas reglas que dependen del tiempo (vencimientos, ventanas de 24 horas, meses cerrables).

2.3 Diseño guiado por el dominio (Eric Evans)

- **Contextos acotados**: acceso, clientes, operación, membresías, evaluación, biblioteca y novedades. Cada uno tiene su modelo y sus tablas.
- **Lenguaje ubicuo**: el código usa las palabras del gimnasio en español (franja, cuadre, pilar, línea base, carga inicial, rutina, plan de trabajo), las mismas que en la interfaz y en el manual.
- **Objetos de valor**: *Dinero* (centavos enteros), *Fecha* (día civil sin hora), *Porcentaje* (puntos base). Son inmutables y validan su propia consistencia.
- **Configuración como datos**: pilares, tests, rúbricas, divisiones de peso y golpes viven en catálogos de la base; el mismo sistema puede servir a otra disciplina.

2.4 Principios SOLID y código limpio

Principio	Aplicación en el sistema
Responsabilidad única	Un caso de uso por intención del usuario (RegistrarCliente, RevisarPago, ReiniciarPuntoDePartida, AplicarComoCoach).
Abierto/cerrado	Nuevas disciplinas, tests o tipos de novedad se agregan como datos o como nuevos casos de uso, sin modificar los existentes.
Sustitución de Liskov	Cualquier implementación de un puerto (Postgres, memoria) cumple el mismo contrato; las pruebas usan ambas.
Segregación de interfaces	Puertos pequeños y específicos (por ejemplo, RepositorioInscripciones separado de RepositorioClientes).
Inversión de dependencias	Los casos de uso reciben sus dependencias por constructor; un contenedor de composición (contenedor.ts) las conecta.
Código limpio	Nombres que revelan intención, funciones cortas, comentarios que explican el porqué, mensajes de error escritos para el usuario final.

2.5 Separación entre comandos y consultas (CQS/CQRS ligero)

Las **modificaciones** pasan por casos de uso que validan permisos, aplican reglas y auditan. Las **lecturas** para pantallas usan objetos de consulta (*ConsultasClientes*, *ConsultasEvaluacion*) que devuelven exactamente lo que la vista necesita. Así las reglas no se contaminan con necesidades de presentación y las consultas se optimizan sin riesgo.

2.6 Multi-inquilino (multi-tenancy) con esquema compartido

Se eligió el patrón de **esquema compartido con columna de inquilino** (*gimnasio_id*) reforzado con **Row Level Security** de PostgreSQL: cada transacción fija el gimnasio y la base sólo deja ver y escribir sus filas. Es la opción de menor costo operativo para muchos gimnasios pequeños, con aislamiento verificado por la propia base de datos y no sólo por el código.

2.7 Seguridad: defensa en profundidad y mínimo privilegio

- **Defensa en profundidad**: permisos revisados en el servidor, RLS en la base, cifrado de campos sensibles, auditoría inmutable y encabezados HTTP de protección. Si una capa falla, otra contiene el daño.
- **Mínimo privilegio**: la app se conecta con un rol (*app_gimnasio*) que no puede saltarse RLS, alterar el esquema, borrar ni modificar la auditoría; las migraciones usan otro rol.
- **Control de acceso basado en roles (RBAC)**: rol → permisos finos (por ejemplo *pago.confirmar*, *sistema.administrar*); las pantallas y los casos de uso preguntan por permisos, no por roles.
- **OWASP Top 10** como lista de verificación: control de acceso roto, fallas criptográficas, inyección (consultas parametrizadas), diseño inseguro, autenticación (Argon2id, freno de intentos), registro y monitoreo (auditoría).

2.8 Criptografía aplicada

- **Argon2id** (ganador de la Password Hashing Competition, recomendado por OWASP) para contraseñas y PIN: resistente a ataques con GPU por ser costoso en memoria.
- **AES-256-GCM** (cifrado autenticado) para notas médicas y contacto de emergencia: confidencialidad e integridad; la clave vive en Secret Manager, fuera de la base.
- **Tokens aleatorios de alta entropía** para sesiones y enlaces, guardados sólo como huella SHA-256: una fuga de la base no permite suplantar sesiones.
- **Comparación en tiempo constante** y hash de relleno cuando un usuario no existe, para no revelar qué cuentas existen por diferencias de tiempo.

2.9 Privacidad por diseño

Se aplica minimización de datos (la clase de prueba sólo pide nombre y WhatsApp), consentimiento explícito para contactar, finalidad declarada en el propio formulario, separación de datos de salud (cifrados y visibles sólo para coach y administración) y vistas agregadas sin datos personales para el superadministrador, en línea con la LFPDPPP mexicana.

2.10 Pirámide de pruebas y desarrollo guiado por pruebas

Muchas pruebas rápidas del dominio puro, menos pruebas de casos de uso con dobles de prueba, y pruebas de integración contra un PostgreSQL real embebido (PGlite) que ejecutan las migraciones y corren **con el mismo rol restringido de producción**: si una consulta olvida fijar el gimnasio, la prueba falla. Las reglas críticas (índices, comisiones, cuadre) se escribieron junto con sus casos de prueba.

2.11 Entrega continua, infraestructura inmutable y metodología Twelve-Factor

- Cada versión es una imagen de contenedor inmutable con su propia revisión en Cloud Run; regresar es mover el tráfico, sin reconstruir.

- Configuración por variables de entorno y secretos administrados; procesos sin estado; registros a la salida estándar; misma imagen en todos los ambientes.
- Migraciones de base **hacia adelante y compatibles** con la versión anterior (expandir y luego contraer), porque durante el despliegue conviven ambas.
- **Versionado semántico** (X.Y.Z) y registro de cambios legible por el usuario (Keep a Changelog).

2.12 Patrones de dominio financiero

- **Patrón Money** (Martin Fowler): el dinero se guarda en centavos enteros, nunca con punto flotante, para evitar errores de redondeo.
- **Registros no destructivos**: pagos, evaluaciones y acuerdos de comisión no se borran; se rechazan, anulan o cierran con motivo. La historia es auditable.
- **Vigencia temporal** (efectividad por fechas): comisiones y cuota del sistema tienen *vigente desde*; un cambio nunca se aplica hacia atrás.

2.13 Diseño de interacción

- **Diseño móvil primero** y **revelación progresiva** (menú agrupado, formularios desplegables, detalles bajo demanda).
- **Heurísticas de Nielsen**: visibilidad del estado (por aprobar, por vencer), prevención de errores (validaciones, confirmaciones), lenguaje del usuario y ayuda contextual.
- **Ley de Fitts** y guías de accesibilidad: objetivos táctiles de al menos 44 px, contraste suficiente sobre fondo oscuro y etiquetas reales en cada control.

3. Requisitos y atributos de calidad

3.1 Requisitos funcionales principales

Área	Requisitos
Acceso	Cuentas personales; entrada por usuario corto o correo; enlaces de activación de un uso; roles combinables; sesiones revocables.
Captación	Portada con marca por gimnasio; clase de prueba; solicitud para ser coach; registro de cliente con invitación o elección de coach.
Operación	Días y franjas; asignación de coaches por turno; coach y franja por cliente con historial; perfil del cliente.
Membresías y dinero	Planes; pagos reportados y confirmados con comprobante; carga inicial; comisiones con vigencia; cuadre mensual con coaches; cuota del sistema.
Evaluación	Sesiones con PIN, check-in de aptitud, tests con intentos, rúbrica de estrategia, índices por pilar contra línea base, alertas, anulación y reinicio de punto de partida.
Entrenamiento	Glosario de golpes, biblioteca de ejercicios con formalización y fusión, rutinas por bloques, plan de trabajo mensual con aprobación.
Comunicación	Novedades con audiencias, eventos con cupo e inscripción, felicitaciones con menciones, novedades públicas.
Sistema	Panel del superadministrador con vista agregada de todos los gimnasios.

3.2 Atributos de calidad (ISO/IEC 25010)

Atributo	Cómo se atiende
Seguridad	RBAC + RLS + cifrado de campos + auditoría inmutable + Argon2id + freno de intentos + encabezados HTTP.
Mantenibilidad	Capas con regla de dependencia verificada; módulos por contexto; tipos estrictos; pruebas automatizadas.
Fiabilidad	Transacciones por operación; restricciones CHECK y llaves foráneas en la base; respaldo nocturno; reversión por revisión.
Usabilidad	Móvil primero, lenguaje del gimnasio, ayuda por rol, mensajes de error accionables.
Portabilidad	Contenedor estándar; PostgreSQL estándar; disciplina configurable por datos.
Eficiencia	Consultas a la medida de cada vista; escala a cero en Cloud Run; base que se suspende sin uso (Neon).

4. Arquitectura

4.1 Vista de capas

```
src/
  shared/
    domain/          Fecha, Dinero, Porcentaje, errores de dominio
    application/     Puertos comunes: Autorizacion, Auditoria, Reloj, GeneradorDeIds
    infrastructure/  Conexión a Postgres, enGimnasio(), auditoría, cifrado
  modules/<módulo>/
    domain/          Entidades y reglas puras (sin dependencias externas)
    application/     Casos de uso y sus puertos
    infrastructure/  Repositorios y consultas en Postgres
  app/               Next.js: páginas, acciones de servidor, API (capa de marco)
    _servidor/       Contenedor de composición, sesión, marca del gimnasio
    _ui/             Componentes de interfaz compartidos
db/migrations/       SQL versionado, aplicado en orden
```

4.2 Flujo de una operación

1. El usuario envía un formulario; React lo manda a una **acción de servidor** (no hay API pública que exponer).
2. La acción obtiene la **sesión** (cookie httpOnly → huella → contexto con roles y permisos) y exige el permiso de la pantalla.
3. El **contenedor** construye el caso de uso con sus adaptadores (repositorio Postgres, reloj del sistema, auditoría).
4. El **caso de uso** valida el permiso fino, aplica las reglas del dominio y llama al repositorio.
5. El repositorio abre una transacción con **enGimnasio()**, que fija el gimnasio para RLS, y ejecuta SQL parametrizado.
6. Se registra la **auditoría** (quién, qué, antes y después) y la acción revalida la pantalla.

4.3 Decisiones de arquitectura relevantes

Decisión	Alternativa descartada	Razón
Monolito modular en Next.js	Microservicios / API separada (NestJS en el plan inicial)	Un equipo pequeño y un dominio cohesivo; menos piezas que operar. Los módulos permiten separar después si hace falta.
Acciones de servidor	API REST pública	Menor superficie de ataque; tipos de extremo a extremo; protección CSRF integrada en el marco.
SQL explícito (postgres.js)	ORM	Control total de RLS, transacciones y consultas; sin capa mágica entre el dominio y la base.
RLS en la base	Filtro sólo en el código	Un olvido en el código no expone datos de otro gimnasio.
Neon + Cloud Run	Servidor virtual propio	Pago por uso, escala a cero, respaldos y restauración instantánea administrados.

5. Módulos y modelo del dominio

Módulo	Responsabilidad	Casos de uso representativos
acceso	Cuentas, credenciales, sesiones, enlaces de activación, equipo, solicitudes de coach	IniciarSesion, UsarEnlaceDeAcceso, AgregarMiembro, CambiarMiUsuario, AplicarComoCoach, AtenderSolicitudCoach
clientes	Registro de clientes, aprobación, inscripción con la propia cuenta, clase de prueba	RegistrarCliente, RevisarRegistroCliente, InscribirmeComoCliente, SolicitarClaseDePrueba, AtenderSolicitud
operacion	Días, franjas, horario de coaches, asignación coach–cliente, perfiles	AsignarTurno, QuitarTurno, AsignarCliente, ActualizarPerfilCliente (con cifrado)
membresias	Planes, pagos, comisiones, cuadre con coaches, cuota del sistema, pesajes	RegistrarPago, RevisarPago, CambiarComision, CerrarCuadre, CambiarTarifaSistema
evaluacion	Sesiones, check-in, captura de tests, rúbrica, índices, línea base, PIN	IniciarEvaluacion, RegistrarChequeo, GuardarCaptura, CerrarEvaluacion, AnularEvaluacion, ReiniciarPuntoDePartida
biblioteca	Golpes y alias, ejercicios, formalización y fusión, rutinas, planes de trabajo	ProponerEjercicio, FormalizarEjercicio, FusionarEjercicio, GuardarRutina, GuardarPlanDeTrabajo, RevisarPlan, MantenerPlan
novedades	Publicaciones por audiencia, eventos con cupo, menciones, lectura	Publicar, Archivar, Inscribirse, CancelarInscripcion

Los módulos no leen tablas ajenas: cuando uno necesita algo de otro, lo pide por un puerto o por una consulta del módulo dueño. Las entidades centrales (gimnasio, usuario, auditoría) viven en *shared*.

6. Modelo de datos y persistencia

Migración	Tablas que introduce
0001 base	disciplina, gimnasio, funcion_gimnasio, usuario, permiso, rol, rol_permiso, usuario_rol, coach, dia_operacion, franja_horaria, acuerdo_comision, auditoria
0002 semilla	Datos del gimnasio piloto, roles y permisos, coaches iniciales
0003 acceso	sesion, enlace_acceso, intento_acceso, cliente
0004 rol de la app	Rol app_gimnasio con mínimo privilegio
0005 operación	asignacion_horario, asignacion_coach_cliente, perfil_cliente
0006 peso y pagos	division_peso, medicion_corporal, plan_membresia, pago, cuadre_coach, tarifa_sistema
0007 evaluación	pilar, test_def, rubrica_criterio, referencia_nivel, bateria, bateria_test, sesion_evaluacion, resultado_intento, resultado_rubrica, linea_base, puntaje_test, puntaje_pilar
0008 biblioteca	golpe, golpe_alias, categoria_ejercicio, ejercicio, ejercicio_alias, rutina, rutina_bloque, bloque_elemento, plan_trabajo, plan_turno
0009 novedades	publicacion, mencion, inscripcion_evento
0010 usuario corto	Usuario corto único por persona
0011 reinicio y prueba	reinicio_linea_base, solicitud_prueba
0012 marca	Nombre, eslogan, frase y logotipo por gimnasio
0013 solicitud de coach	solicitud_coach

Principios de persistencia

- **Integridad declarativa**: llaves foráneas, CHECK (formatos de teléfono, longitudes, estados válidos) e índices únicos parciales; la base rechaza datos inválidos aunque el código fallara.
- **Migraciones inmutables**: una migración aplicada nunca se edita; cada cambio es una nueva, registrada en *schema_migraciones*.
- **Instantáneas de cálculo**: los puntajes por test y por pilar se guardan al publicar la evaluación, para que cambios futuros de fórmula no reescriban el pasado.
- **Tiempo**: instantes en *timestamptz* (UTC) y días civiles calculados en la zona horaria del gimnasio (America/Mexico_City).

7. Seguridad y privacidad

Capa	Medida
Identidad	Argon2id para contraseñas y PIN; mínimo 10 caracteres; 5 intentos en 15 min frenan la cuenta; enlaces de activación de un uso que vencen en 72 h.
Sesión	Token aleatorio en cookie httpOnly, Secure y SameSite=Lax, 30 días; en la base sólo su huella SHA-256; cierre de sesión que la revoca.
Autorización	RBAC con permisos finos revisados en la acción de servidor y otra vez en el caso de uso. El rol de superadministrador no se puede otorgar desde la app.
Aislamiento	Row Level Security por gimnasio en todas las tablas con gimnasio_id; la app usa un rol sin BYPASSRLS. Funciones SECURITY DEFINER acotadas para lo público.
Datos sensibles	Notas médicas y contacto de emergencia cifrados con AES-256-GCM; clave en Secret Manager; nunca se auditan en claro.
Auditoría	Tabla de sólo inserción (un disparador impide modificarla o borrarla): quién, cuándo, acción, entidad, antes y después.
Comprobantes	Archivos privados; sólo los ve la administración y quien reportó el pago; nunca hay enlace público.
Formularios públicos	Campo trampa contra robots, deduplicación por teléfono en 24 h, consentimiento explícito, validación en servidor y en la base.
Transporte y navegador	HTTPS obligatorio (HSTS), X-Frame-Options DENY, X-Content-Type-Options nosniff, Referrer-Policy estricta.
Secretos	Cadenas de conexión y clave de datos en Google Secret Manager; .env.local nunca se sube al repositorio.

8. Reglas del negocio y fundamentos del dominio deportivo

8.1 Evaluación por pilares: medir el avance individual

El modelo mide el **progreso de cada persona contra su propio punto de partida**, no contra una norma poblacional. Esto permite comparar coaches por el avance que logran (un principiante que mejora cuenta igual que un avanzado) y motiva al cliente con un número claro.

- **Línea base = 100**: la primera evaluación válida fija la base de cada test.
- **Normalización**: si mayor es mejor, índice = actual ÷ base × 100; si menor es mejor (tiempos), índice = base ÷ actual × 100. Sin base válida no hay índice.
- **Rúbrica de estrategia** (táctica y adaptabilidad, conductas 0–1–2, total 0–10): índice = 100 + (actual − base) × 10, porque la base puede ser 0.
- **Pilar** = promedio ponderado de sus tests; si falta un test obligatorio, el pilar no se publica. Estrategia = 50% táctica + 50% adaptabilidad.
- **Detección de atípicos**: un cambio mayor a ±40% contra la evaluación anterior se señala como probable error de captura.
- **Validez**: check-in de aptitud (con lesión o dolor no se evalúa), misma báscula, intentos repetidos y nulos explícitos; PIN para que sólo evalúe la administración.
- **Reinicio de punto de partida** tras ausencia larga o lesión: la siguiente evaluación vuelve a ser la inicial sin borrar la historia.

8.2 Composición corporal

Las divisiones de peso siguen las categorías de World Boxing por rama; el IMC se muestra sólo como referencia. La división vigente es la del último pesaje.

8.3 Membresías, comisiones y cuadre

- La membresía corre desde la fecha del pago; si aún había días, se encadena al terminar. Por vencer = 5 días o menos.
- Comisión del coach con vigencia por fechas: se usa el acuerdo vigente el día de la confirmación (excepción del coach o general).
- Cuadre mensual por coach: comisiones del mes contra el dinero que él cobró; saldo positivo lo paga la administración, negativo lo entrega el coach. Sólo se cierran meses terminados.
- La **carga inicial** (clientes que ya entrenaban) activa la membresía pero no genera comisión ni cuenta como ingreso.
- Cuota del sistema: porcentaje de los ingresos o cuota fija, sólo programable por el superadministrador desde un mes futuro.

9. Interfaz y experiencia de uso

- Modo oscuro con acento rojo, números grandes y tarjetas: legible en el piso del gimnasio.
- Marca por gimnasio (logotipo, nombre y eslogan en letras góticas) leída de la base en cada petición.
- Menú agrupado por rol con segunda fila contextual; un solo nivel para quien tiene un rol.
- Formularios con estado de envío, mensajes de error en lenguaje claro y conservación de lo escrito si algo falla.
- Ayuda por rol dentro de la app, alineada con el manual de uso.
- Tipografías: Barlow Condensed (títulos), Inter (texto) y Grenze Gotisch (marca), servidas por el propio sitio.

10. Calidad y pruebas

Nivel	Qué cubre	Herramienta
Dominio	Fechas, dinero, índices, divisiones, horario, comisiones, cuadre, políticas de acceso	Vitest
Casos de uso	Reglas de permisos y flujos con dobles en memoria y reloj fijo	Vitest
Integración	Migraciones completas y repositorios contra PostgreSQL real embebido, ejecutando como app_gimnasio (RLS activo)	PGlite + pglite-socket
Estático	Tipos estrictos y regla de dependencia entre capas	TypeScript, ESLint
Manual	Recorrido de cada función nueva en el navegador con base local antes de publicar	Base en memoria + servidor de desarrollo

La publicación se niega si falla cualquier verificación. Estado actual: 146 pruebas en 23 archivos, todos en verde.

11. Infraestructura, despliegue y operación

Componente	Servicio	Notas
Aplicación	Google Cloud Run (us-central1)	Contenedor Node 22 en modo standalone, usuario sin privilegios, escala a cero.
Construcción	Cloud Build	Dockerfile de tres etapas (dependencias, construcción, ejecución).
Base de datos	Neon PostgreSQL	Rama production; restauración instantánea a un punto reciente.
Secretos	Secret Manager	database-url, database-url-app, clave-datos.
Archivos	Cloud Storage	Comprobantes privados y respaldos.
Respaldos	Cloud Run Job + Scheduler	Volcado diario 3:00 (CDMX) comprimido, retención 90 días + 30 de borrado suave.
Salud	/api/salud	Versión, commit, estado de la base y número de migraciones; se verifica en cada publicación.
Costos	Alerta de presupuesto	100 MXN al mes en la etapa piloto.

Publicación (npm run publicar)

1. Exige cambios guardados, entrada en el CHANGELOG y etiqueta vX.Y.Z en el commit.
2. Corre lint, tipos y pruebas.
3. Aplica las migraciones pendientes en Neon con el rol dueño.
4. Despliega en Cloud Run con revisión gimnasio-boxeo-vX-Y-Z (una versión no se puede repetir).
5. Verifica /api/salud con la versión esperada y sube commit y etiqueta a GitHub.

Reversión: mover el tráfico a la revisión anterior, sin reconstruir. Restauración de datos: un respaldo se carga en una rama nueva de Neon, nunca directo sobre producción.

12. Gestión de la configuración y versiones

- Git con rama principal y etiquetas anotadas por versión; mensajes de commit en español.
- Versión semántica: Z para correcciones, Y para funciones nuevas o fases, X para cambios mayores (1.0.0 será el piloto en uso real).
- CHANGELOG escrito en palabras del usuario (Agregado / Cambiado / Corregido).
- Cambios de base de datos siempre como migración nueva, compatible con la versión que sigue corriendo durante el despliegue.

13. Decisiones, riesgos y deuda técnica

Tema	Situación	Plan
Segundo factor (2FA)	Previsto en el plan para administración y superadmin; aún no implementado.	Agregar TOTP antes de abrir a otros gimnasios.
Clave de datos	Si se pierde, los datos cifrados no se recuperan.	Nunca reemplazarla ni borrarla; documentado en README y CLAUDE.md.
Panel del sistema	Versión inicial de sólo lectura.	Cobros del servicio, métricas por pilar y alta de gimnasios.
Ayuda en código	Las guías viven en el código.	Pasarlas a tablas editables (guia, guia_paso).
Un solo gimnasio público	La portada usa un gimnasio fijo.	Dirección propia por gimnasio (subdominio o ruta).
Dependencia de proveedores	Neon y Google Cloud.	SQL y contenedor estándar: portables a otro proveedor.

14. Glosario

Término	Significado
Caso de uso	Una intención del usuario implementada como clase con sus reglas (por ejemplo, RevisarPago).
Puerto / adaptador	Interfaz que pide un caso de uso / su implementación concreta (Postgres, memoria).
RLS	Row Level Security: la base filtra filas según el gimnasio de la transacción.
Línea base	Primera evaluación válida de cada test; vale 100.
Pilar	Fuerza, velocidad, resistencia o estrategia; promedio ponderado de sus tests.
Franja	Bloque de 2 horas del horario (8–10, 10–12, ... 20–22).
Cuadre	Conciliación mensual de comisiones contra cobros, por coach; y del gimnasio con el sistema.
Carga inicial	Pago vigente de un cliente que ya entrenaba antes del sistema.
Revisión	Versión desplegada e inmutable de la app en Cloud Run.